

Week4: SNN 在线训练方法

OTTT 与 NDOT 讲稿

Tao

摘要

本文为 Week4 “SNN Training” 课程的讲稿内容，对应于 Week4_SNN Training.pptx。在前三周已讲授 SNN 前向动力学、BPTT 与代理梯度的基础上，本周系统介绍从 BPTT 到在线训练的方法论跃迁，重点讲述 OTTT (Online Training Through Time) 和 NDOT (Neuronal Dynamics-based Online Training) 两种代表性在线训练算法，包括其数学推导、核心近似、递推实现及对比分析。

RTRL: A learning algorithm for continually running fully recurrent neural networks [1];

E-prop: A solution to the learning dilemma for recurrent networks of spiking neurons [2];

OTTT: Online Training Through Time for Spiking Neural Networks [3];

NDOT: NDOT: Neuronal Dynamics-based Online Training for Spiking Neural Networks [4];

S-TLLR: S-TLLR: STDP-inspired Temporal Local Learning Rule for Spiking Neural Networks [5];

TESS: TESS: A Scalable Temporally and Spatially Local Learning Rule for Spiking Neural Networks [6]

I. 课程回顾与本周概述

前情回顾

在前三周的课程中，我们已经讲授了以下内容：

- **Week 1:** SNN 的基本概念——LIF 神经元模型、脉冲生成机制、与 ANN 的核心差异。
- **Week 2:** BPTT (Backpropagation Through Time) 的完整推导——将 SNN 沿时间展开为深度计算图，利用链式法则计算梯度。核心结论：BPTT 梯度精确，但需要存储全部时间步的中间状态 ($O(T)$ 内存)，且无法在线更新。
- **Week 3:** STBP (Spatio-Temporal Backpropagation) 与代理梯度 (Surrogate Gradient)——用光滑函数 (如矩形函数、sigmoid 函数) 的导数替代不可导的 Heaviside 阶跃函数导数，使梯度能够传播。代理梯度的引入使得 BPTT 在 SNN 中实际可用，但未能解决内存和在线更新的瓶颈。

本周内容

- **核心问题：**如何在保持梯度精度的前提下，实现 $O(1)$ 内存的在线训练？
- **方法论主线：**

BPTT (离线, $O(T)$) $\longrightarrow$ OTTT (在线, $O(1)$, 近似) $\longrightarrow$ NDOT (在线, $O(1)$, 物理)

- **关键思想：**将 BPTT 中需要时间回溯的连乘项 $\prod \epsilon[i]$ ，简化为可前向递推的衰减累积，从而打破 $O(T)$ 内存限制。

a) 讲稿：：今天我们进入第四周的课程——SNN 的在线训练方法。前三周我们打下了扎实的基础：从 LIF 神经元、到 BPTT 的完整推导、再到代理梯度。今天我们要解决的核心问题是：BPTT 虽然精确，但内存随序列长度线性增长，这对边缘设备和神经形态芯片来说是不可接受的。那么，有没有办法让 SNN 逐个时间步地在线学习、内存不随时间增长呢？答案是有的——OTTT 和 NDOT 就是两种代表性的解决方案。

II. BPTT 的核心瓶颈回顾

LIF 神经元离散动力学

在进入在线方法之前，我们先回顾 BPTT 的核心公式。LIF 神经元的离散动力学为：

$$\mathbf{u}^l[t+1] = \lambda(\mathbf{u}^l[t] - V_{th}\mathbf{s}^l[t]) + \mathbf{W}^l\mathbf{s}^{l-1}[t+1] \quad (1)$$

其中：

- $\mathbf{u}^l[t]$ ：第 l 层 t 时刻的膜电位
- $\mathbf{s}^l[t] = H(\mathbf{u}^l[t] - V_{th})$ ：脉冲输出（Heaviside 阶跃函数）
- λ ：泄漏常数（ $0 < \lambda \leq 1$ ）
- V_{th} ：发放阈值
- $\mathbf{W}^l$ ：待训练的权重矩阵

BPTT 完整梯度公式

BPTT 对权重 $\mathbf{W}^l$ 的梯度为：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \left(\frac{\partial \mathbf{u}^{l+1}[t]}{\partial \mathbf{W}^l} + \sum_{\tau < t} \prod_{i=t-1}^{\tau} \left(\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \right) \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right) \quad (2)$$

BPTT 的两大瓶颈

- 1) **内存瓶颈：**连乘项 $\prod_{i=t-1}^{\tau}$ 需要存储全部 T 个时刻的 $\mathbf{u}$ 和 $\mathbf{s}$ ，内存复杂度 $O(T)$ 。
- 2) **离线瓶颈：**必须等待整个长度为 T 的序列处理完毕才能计算梯度并更新权重，无法实现在线（流式）学习。

这两大瓶颈的根本原因都指向同一个结构——公式 (2) 中橙色的连乘项，它要求我们沿着时间轴反向追溯，访问所有历史状态。

a) 讲稿：：我们先来回顾 BPTT 的梯度公式。注意看这个公式的结构：外层是对时间 t 的求和，这是因为权重在所有时间步共享。括号里有两项——当前直连项和历史回溯项。当前直连项很简单，等于当前输入脉冲 $\mathbf{s}^l[t]$ 。问题出在历史回溯项，它包含一个从 τ 到 $t-1$ 的连乘。这个连乘里有两部分：绿色的是泄漏路径 λ ，青色 $\times$ 红色的是重置路径。因为连乘涉及整个历史区间，反向传播时必须把所有中间状态存在内存里，这就导致了 $O(T)$ 的内存复杂度。今天我们要讲的所有方法，本质上都是在“如何打破这个连乘依赖”上做文章。

III. 从 BPTT 到在线学习：核心思路

为什么连乘是瓶颈？

BPTT 梯度中的连乘结构：

$$\prod_{i=\tau-1}^{\tau} \epsilon[i], \quad \epsilon[i] = \lambda + (-\lambda V_{th}) \frac{\partial s}{\partial u} \quad (3)$$

这个结构在反向传播时天然需要访问所有中间状态——这是一个因果链：要计算 t 时刻的梯度，必须通过每一个中间时间步反向传递误差信号。

核心突破点

关键在于观察：

$$\frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} = \mathbf{s}^l[\tau]$$

即膜电位对权重的直接偏导恰好等于输入脉冲。如果时序核能够退化为一个纯衰减因子，那么历史贡献就变成了：

$$\sum_{\tau \leq t} \lambda^{t-\tau} \mathbf{s}^l[\tau]$$

这个式子可以通过前向递推高效计算：

$$\hat{\mathbf{a}}^l[t] = \lambda \hat{\mathbf{a}}^l[t-1] + \mathbf{s}^l[t]$$

只需要维护一个大小为 n 的向量 $\hat{\mathbf{a}}$ ，内存从 $O(T)$ 骤降至 $O(1)$ ！

a) 讲稿：：大家可能已经注意到了，BPTT 的瓶颈完全集中在那个时间连乘上。如果我们能把这个连乘简化，是不是就能实现在线学习呢？这个思路就是 OTTT 的出发点。我们注意到一个关键的恒等式：膜电位对权重的偏导正好是输入脉冲。如果时序核能退化成简单的指数衰减 $\lambda^{t-\tau}$ ，那么整个历史回溯项就变成了一个前向可递推的量 $\hat{\mathbf{a}}[t]$ 。这就把 $O(T)$ 的内存需求削减到了 $O(1)$ 。关键问题是：怎么处理那个复杂的 $\epsilon[i]$ ，让它退化成 λ ？

IV. OTTT：核心近似与推导

OTTT 的核心近似

OTTT 的核心理念非常简洁：在时序连乘中，不对红色项施加代理梯度，而是直接取其真实导数（几乎处处为 0）：

$$\frac{\partial \mathbf{u}[i+1]}{\partial \mathbf{s}[i]} \frac{\partial \mathbf{s}[i]}{\partial \mathbf{u}[i]} \approx 0 \quad \Rightarrow \quad \prod_{i=\tau}^{t-1} (\lambda \mathbf{I} + 0) = \lambda^{t-\tau} \mathbf{I} \quad (4)$$

简化后的梯度公式

经过 OTTT 近似后，BPTT 的梯度退化为：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \left(\sum_{\tau \leq t} \lambda^{t-\tau} \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right) \quad (5)$$

注意，红色项仅在空间传播入口处保留（仍需代理梯度），但在时序回溯中已被完全消除。

定义追踪变量

定义突触前活动踪迹：

$$\hat{\mathbf{a}}^l[t] \triangleq \sum_{\tau=1}^t \lambda^{t-\tau} \mathbf{s}^l[\tau] \quad (6)$$

利用 $\frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} = \mathbf{s}^l[\tau]$ ，梯度可重写为：

$$\nabla_{\mathbf{W}^l} L = \sum_{t=1}^T \mathbf{g}_{\mathbf{u}^{l+1}}[t] (\hat{\mathbf{a}}^l[t])^\top \quad (7)$$

其中 $\mathbf{g}_{\mathbf{u}^{l+1}}[t] = \left(\frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \right)^\top$ 为空间梯度。

前向递推（在线计算的关键）

$\hat{\mathbf{a}}^l[t]$ 可以通过前向递推在线计算：

$$\hat{\mathbf{a}}^l[t] = \lambda \hat{\mathbf{a}}^l[t-1] + \mathbf{s}^l[t] \quad (8)$$

这一递推仅依赖上一时刻的 $\hat{\mathbf{a}}$ 和当前时刻的脉冲，完全不需要存储或访问更早期的历史状态。

a) 讲稿： ∵ OTTT 的核心近似非常大胆——它直接说“重置路径那一项（青色×红色）对时序传播的贡献可以忽略不计”。为什么可以这样近似？因为红色项 $\partial s / \partial u$ 的真实导数几乎处处为零，只在阈值附近有一个无穷大的尖峰。既然我们本来就用代理梯度去近似它，那在时间传播中干脆把它置零，只保留确定性的泄漏项 λ ，这不是更一致吗？

这个近似带来了巨大的实际好处。连乘退化为 $\lambda^{t-\tau}$ ，是一个固定的指数衰减，与神经元的具体发放状态无关。然后我们定义 $\hat{a}[t]$ 为历史脉冲的指数加权和——这恰好就是突触前活动的“资格迹”。最关键的是， $\hat{a}[t]$ 可以用递推式 $\hat{a}[t] = \lambda \hat{a}[t-1] + s[t]$ 前向计算，只需要存一个向量，内存 $O(1)$ ，完美实现在线学习。

最终梯度变成了空间梯度与追踪变量的外积 $\mathbf{g} \cdot \hat{\mathbf{a}}^\top$ ，形式和 BPTT 一致，但历史信息全部被 $\hat{a}$ 捕获，不再需要反向时间传播。

表 I: BPTT vs OTTT 对比

特性	BPTT	OTTT
内存复杂度	$O(T)$	$O(1)$
在线更新	否	是
时间传播核	$\prod(\lambda + \text{reset})$	$\lambda^{t-\tau}$
代理梯度使用	时间+空间	仅空间
追踪变量	无	$\hat{a}[t] = \lambda \hat{a}[t-1] + s[t]$

V. OTTT 的简化效果与局限分析

OTTT 的简化效果

OTTT 的局限性

尽管 OTTT 在工程上取得了巨大成功（首次实现了 ImageNet 级别的 SNN 在线训练），但它的近似过于粗暴：

- 1) **固定衰减 λ** ：所有神经元在任何时刻都使用相同的衰减常数，但真实神经元中，膜电位的衰减受充电、泄漏、发放、重置的共同影响，是一个动态过程。
- 2) **忽略重置路径**：当神经元发放 spike 后，膜电位会被重置 ($u \leftarrow u - V_{th}$)，这一突变会影响后续所有时间步的膜电位动态。OTTT 完全忽略了这个影响。
- 3) **精度损失**：在需要精确时序信息的任务上（如 DVS 事件流处理、时序分类），固定衰减的近似可能导致精度下降。

a) 讲稿：：OTTT 实现了从 $O(T)$ 到 $O(1)$ 的飞跃，这是一个很大的工程贡献。但我们需要诚实地面对它的局限：所有神经元用同一个固定的 λ ，这意味着无论神经元当前处于什么状态——正在充电、刚刚放电、还是处于静息——它的“历史衰减”都是一样的。这显然不符合真实的神经元动力学。那么，有没有办法用一个动态的、依赖神经元状态的衰减系数来替代固定的 λ 呢？这就是 NDOT 要做的事情。

VI. NDOT：基于神经元动力学的重参数化

从连续 LIF 方程出发

NDOT 的核心思想是不再删除任何项，而是利用神经元自身的物理方程来推导时序核。我们从连续时间的 LIF 微分方程开始：

$$u'(t) = -u(t) + I(t) \quad (9)$$

其中 $I(t)$ 为输入电流。这个方程描述了膜电位在连续时间下的演化——泄漏 ($-u(t)$) 和充电 ($I(t)$)。

定义动态时序依赖 $e(t)$

NDOT 定义了一个核心量——相邻时刻膜电位的变化率之比：

$$e(t) \triangleq \frac{\partial u(t+1)}{\partial u(t)} = \frac{u'(t+1)}{u'(t)} \quad (10)$$

这一步的直觉是：如果我们能知道膜电位在两个相邻时刻之间的变化率之比，就能知道前一时刻的状态对后一时刻的影响有多大。

消去输入电流 $I(t)$

将 LIF 方程代入：

$$e(t) = \frac{-u(t+1) + I(t+1)}{-u(t) + I(t)} \quad (11)$$

离散化并化简

在离散 LIF 动力学中，输入电流满足：

$$I[t+1] = u[t+1] - \lambda(u[t] - V_{th}s[t])$$

代入并化简，得到 NDOT 的核心闭式解：

$$\boxed{\mathbf{e}^l[t] = \frac{\mathbf{u}^l[t] - V_{th}\mathbf{s}^l[t]}{\mathbf{u}^l[t-1] - V_{th}\mathbf{s}^l[t-1]}} \quad (12)$$

这个表达式的物理含义非常清晰：它是两个相邻时刻“重置后膜电位”的比值。分子和分母分别是当前时刻和前一时刻发放重置后的残余膜电位。注意到，它完全不包含代理梯度——纯粹由前向动力学状态决定。

a) 讲稿：：NDOT 的思路和 OTTT 完全不同。OTTT 是“我删掉一项”，NDOT 是“我用物理来重新推导这一项”。NDOT 从连续时间的 LIF 微分方程出发： $u'(t) = -u(t) + I(t)$ 。它没有用 λ 作为固定的离散衰减，而是定义 $e(t) = u'(t+1)/u'(t)$ ——即膜电位变化率的比值——来衡量相邻时刻的依赖性。

关键的推导技巧是利用隐函数定理：在连续系统中， $u(t+1)$ 对 $u(t)$ 的偏导等于两者对时间导数之比。然后把 LIF 方程代入，消去输入电流 $I(t)$ ，再离散化代入离散 LIF 方程——最终得到了一个极其漂亮的闭式解： $\mathbf{e}[t]$ 等于相邻时刻重置后膜电位的比值。

这个 $e[t]$ 的值会在每个时间步、对每个神经元独立计算。如果某个神经元刚刚发放，它的 $e[t]$ 会比 λ 小；如果神经元长时间没有发放， $e[t]$ 接近 λ 。这就捕捉了 OTTT 完全丢失的信息——神经元状态对时序传播的动态影响。

VII. NDOT 的动态时序核与望远镜化简

定义动态时序传播核

与 OTTT 的 $\lambda^{t-\tau}$ 对应，NDOT 定义了动态时序传播核：

$$\mathbf{P}_{k,t}^l \triangleq \prod_{i=k}^{t-1} \mathbf{e}^l[i] = \mathbf{e}^l[k] \odot \mathbf{e}^l[k+1] \odot \cdots \odot \mathbf{e}^l[t-1] \quad (13)$$

其中 $\odot$ 为逐元素乘法（Hadamard product）。

望远镜化简 (Telescoping Cancellation)

这是 NDOT 最优雅的数学性质之一。将 $\mathbf{e}[i]$ 的分式形式代入 $\mathbf{P}_{k,t}$:

$$\begin{aligned} \mathbf{P}_{k,t}^l &= \frac{u[k] - V_{th}s[k]}{u[k-1] - V_{th}s[k-1]} \odot \frac{u[k+1] - V_{th}s[k+1]}{u[k] - V_{th}s[k]} \odot \dots \odot \frac{u[t-1] - V_{th}s[t-1]}{u[t-2] - V_{th}s[t-2]} \\ &= \frac{\mathbf{u}^l[t-1] - V_{th}\mathbf{s}^l[t-1]}{\mathbf{u}^l[k-1] - V_{th}\mathbf{s}^l[k-1]} \end{aligned} \quad (14)$$

中间所有相邻项相互约去，最终只剩下首尾两项的比值！这意味着，虽然 $\mathbf{P}_{k,t}$ 在定义上是一个连乘，实际计算时可以一步到位，无需逐项相乘。

追踪变量的递推

利用 $\mathbf{P}_{k,t}$ 的递推性质，NDOT 同样可以定义前向递推的追踪变量：

$$\hat{\mathbf{a}}^l[t] = \mathbf{e}^l[t-1] \odot \hat{\mathbf{a}}^l[t-1] + \mathbf{s}^l[t] \quad (15)$$

最终梯度公式

与 OTTT 形式完全一致：

$$\nabla_{\mathbf{W}^l} \mathcal{L} = \sum_{t=1}^T \mathbf{g}_{\mathbf{u}^l}[t] (\hat{\mathbf{a}}^{l-1}[t])^\top \quad (16)$$

a) 讲稿：：NDOT 的 $\mathbf{P}_{k,t}$ 是对 OTTT 的 λ^{t-k} 的直接推广——从标量指数衰减推广为逐元素的动态比值连乘。但这里有一个非常漂亮的数学性质：当每个步骤都是分式比值时，连乘会产生望远镜效应——中间所有相邻的分子分母相互约去，最终 $\mathbf{P}_{k,t}$ 仅等于首尾两项的比值。这意味着我们不实际上需要计算那个连乘——计算效率和 OTTT 一样高效。

更进一步，追踪变量的递推形式与 OTTT 完全一致： $\hat{\mathbf{a}}[t] = \mathbf{e}[t-1] \odot \hat{\mathbf{a}}[t-1] + \mathbf{s}[t]$ ，唯一的区别就是把固定的 λ 换成了动态的 $\mathbf{e}[t-1]$ 。最终梯度也是 $\mathbf{g} \cdot \hat{\mathbf{a}}^\top$ 的形式。所以 NDOT 在保持和 OTTT 完全相同的 $O(1)$ 内存和计算复杂度的前提下，用更精确的物理量替换了近似常数。

VIII. OTTT vs NDOT 核心对比

表 II: OTTT 与 NDOT 核心对比

维度	OTTT	NDOT
时序核 (单步)	$\mathbf{e}[i] = \lambda$	$\mathbf{e}[i] = \frac{u[i] - V_{th}s[i]}{u[i-1] - V_{th}s[i-1]}$
历史传播核	λ^{t-k}	$\mathbf{P}_{k,t} = \prod \mathbf{e}[i]$ (望远镜化简)
追踪变量递推	$\hat{\mathbf{a}}[t] = \lambda \hat{\mathbf{a}}[t-1] + \mathbf{s}[t]$	$\hat{\mathbf{a}}[t] = \mathbf{e}[t-1] \odot \hat{\mathbf{a}}[t-1] + \mathbf{s}[t]$
额外存储	无	需存 $u[t] - V_{th}s[t]$
方法论本质	近似删除	物理重参数化
内存复杂度	$O(1)$	$O(1)$

一句话总结

OTTT 是把 BPTT 中的重置路径删掉，用固定 λ 近似时序依赖；NDOT 是用连续的神经元物理方程重新推导时序依赖，用动态比值 $e[t]$ 替代固定 λ ，在不增加计算成本的前提下获得更精确的梯度估计。

a) 讲稿：：现在是总结对比的时候。OTTT 和 NDOT 的外在形式惊人地相似——相同的递推结构、相同的 $O(1)$ 内存、相同的 $\mathbf{g} \cdot \hat{\mathbf{a}}^\top$ 最终形式。但它们的本质差异在于方法论：OTTT 走的是“近似简化”的路线——我把一项设为零，问题就简单了；NDOT 走的是“物理精确化”的路线——我不删除任何东西，而是从连续动力学重新推导出正确的依赖关系。

结果就是，NDOT 在相同的计算成本下获得了更高的精度。在 CIFAR-100 上，NDOT 比 OTTT 提升了大约 5.5 个百分点。这是一个相当大的提升，而且“免费”——没有增加额外的计算负担。

IX. 实验结果对比

OTTT vs NDOT 精度对比

表 III: OTTT vs NDOT 在标准 benchmark 上的精度对比

数据集	OTTT	NDOT
CIFAR-10	93.73%	94.90%
CIFAR-100	71.11%	76.61%
DVS-CIFAR10	—	77.50%

NDOT 在 CIFAR-100 上相比 OTTT 提升约 +5.5%，充分证明了用动态神经元动力学替代固定衰减的有效性。

关键结论

- 1) 在线训练是可行的：OTTT 和 NDOT 都证明了，SNN 可以在 $O(1)$ 内存和逐时间步更新的约束下实现有竞争力的精度。
- 2) 物理精确性带来增益：NDOT 相比 OTTT 的精度提升，完全来自用动态比值 $e[t]$ 替代固定 λ 。这证明了“用物理精确量替换工程近似”这一思路的有效性。
- 3) 在线 ≠ 低精度：NDOT 在 CIFAR-10 上的 94.90% 已经不输于许多 BPTT 训练的 SNN，证明在线训练可以匹敌离线训练的精度。

a) 讲稿：：实验结果一目了然：NDOT 在所有 benchmark 上都优于 OTTT，特别是在 CIFAR-100 上有 5.5 个百分点的提升。我想强调的重点是：这个提升是“免费”的——NDOT 的计算量几乎和 OTTT 一样，只是多存了一个 $u[t] - V_{th}s[t]$ 向量，内存仍然是 $O(1)$ 。这说明“物理精确性”不是一个纯粹的理论卖点，它在实际精度上有切实的回报。同时也证明了在线训练可以做到和 BPTT 一样好——NDOT 在 CIFAR-10 上的 94.90% 已经非常接近 BPTT 的 SOTA 水平。

X. 从在线到本地：方法论延伸

本周总结——我们走到了哪里

OTTT 和 NDOT 解决了 BPTT 的时间维度瓶颈——实现了前向时间学习和 $O(1)$ 内存。

但如果我们审视梯度公式 $\mathbf{g} \cdot \hat{\mathbf{a}}^\top$ ，注意到空间梯度 $\mathbf{g}_{\mathbf{u}^{l+1}}[t] = \left(\frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \right)^\top$ 仍然需要从输出层逐层反向传播——空间维度还没有被“本地化”。

下一站预览

下周我们将介绍从“时间本地”迈向“时空全本地”的方法：

- **S-TLLR**: 基于 STDP 启发，用三因子学习规则（资格迹 + 学习信号）实现时间本地化，但空间仍依赖 BP/DFA。
- **TESS**: 在 S-TLLR 基础上通过 LSG（Local Signal Generation）用固定随机投影在层内生成本地学习信号，彻底切断层间依赖，实现时空全本地化。

a) 讲稿：最后，让我们回顾一下我们现在的位置。OTTT 和 NDOT 在时间维度上已经实现了突破性的进展——从 $O(T)$ 到 $O(1)$ ，从离线到在线。但我们观察最终的梯度形式 $\mathbf{g} \cdot \hat{\mathbf{a}}^\top$ ，会发现空间梯度 $\mathbf{g}$ 仍然需要从输出层传回来——虽然这个传播不涉及时间，但仍然是“全局的”。

在下周的课程中，我们将探索如何把“本地化”从时间维度扩展到空间维度。S-TLLR 用生物学启发的三因子规则来本地化时间信用分配，TESS 则更进一步，用一个固定随机投影在每一层内部直接生成学习信号，完全不需要层间传播。这将是 SNN 训练方法从“在线”到“全本地”的最后一步跳跃。

参考文献

- [1] R. J. Williams and D. Zipser, “A learning algorithm for continually running fully recurrent neural networks,” *Neural computation*, vol. 1, no. 2, pp. 270–280, 1989.
- [2] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, no. 1, p. 3625, Jul. 2020. [Online]. Available: <https://doi.org/10.1038/s41467-020-17236-y>
- [3] M. Xiao, Q. Meng, Z. Zhang, D. He, and Z. Lin, “Online Training Through Time for Spiking Neural Networks,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 20717–20730. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/82846e19e6d42ebfd4ace4361def29ae-Paper-Conference.pdf
- [4] H. Jiang, G. D. Masi, H. Xiong, and B. Gu, “NDOT: Neuronal Dynamics-based Online Training for Spiking Neural...” [Online]. Available: <https://openreview.net/forum?id=eIF0QoBSFV>
- [5] M. P. E. Apolinario and K. Roy, “S-TLLR: STDP-inspired Temporal Local Learning Rule for Spiking Neural Networks,” Oct. 2024, arXiv:2306.15220 [cs]. [Online]. Available: <http://arxiv.org/abs/2306.15220>
- [6] M. P. E. Apolinario, K. Roy, and C. Frenkel, “TESS: A Scalable Temporally and Spatially Local Learning Rule for Spiking Neural Networks,” in *2025 International Joint Conference on Neural Networks (IJCNN)*, Jun. 2025, pp. 1–9, iSSN: 2161-4407. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11227652>